FATIMA JINNAH WOMEN UNIVERSITY

Rawalpindi

DevOps – Lab 06

Artifact Management with Nexus Repository Manager

Name	Mahnoor Sajjad
Roll No.	2023-BCS-054 (Section A)
Program	BS Computer Science
Course	DevOps
Lab	Lab 06 (Nexus Repository Manager)
GitHub Username	mahnoor-cs6767
Repository	https://github.com/mahnoor-cs6767/DevOps

Objective

This lab shows how to use Nexus Repository Manager, running in Docker, as a central place to store build artifacts. You check access to Nexus with a restricted user, publish a Gradle artifact (java-app) and a Maven artifact (java-maven-app) to the maven-snapshots repository, and inspect them through the Nexus web interface and REST API. You then create a separate blob store and a Docker repository that uses it, and finally free disk space with a cleanup policy and a blob store compaction task.

Tools and technologies

- Nexus Repository Community Edition 3.98.0-07, running in the Docker container sonatype/nexus3
- Gradle (maven-publish plugin) and Maven (deploy plugin) to upload artifacts
- curl and the Nexus REST API to query repositories and components
- Docker, to run Nexus and to look inside its storage

Lab overview

Part	Topic	Project	Steps
1	Nexus setup and access	Nexus container	1–5
2	Publish a Gradle artifact	java-app	6–11
3	Publish a Maven artifact	java-maven-app	12–16
4	Blob stores and a Docker repository	Nexus	17–20
5	Cleanup policies and blob store compaction	Nexus	21–27

Part 1: Nexus Repository Manager Setup and Access

Step 1: Check that the Nexus container is running

Nexus Repository Manager is a server that stores build artifacts such as JAR files and Docker images. In this lab it runs in a Docker container named nexus.

What to do

1. List the running containers and filter for Nexus:

```
docker ps | grep nexus
```

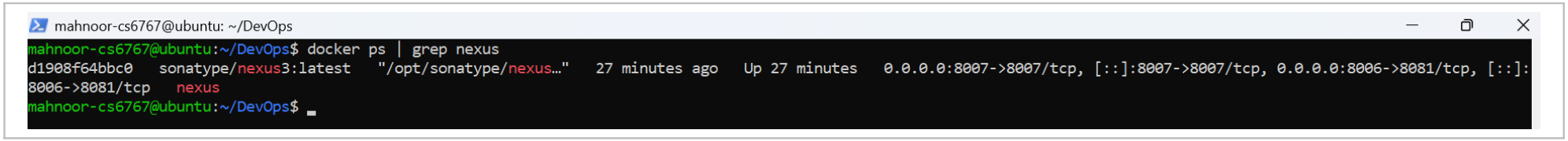


Figure 1: The nexus container is up

What to look for

- The container uses the image sonatype/nexus3:latest and shows “Up 27 minutes”.
- Port 8006 on your machine is mapped to port 8081 inside the container, so the Nexus web interface and API are available at http://localhost:8006.
- Port 8007 is also published. It is used later for the Docker repository.

Step 2: Check that Nexus responds

Send a request to Nexus's status endpoint to confirm that the server is reachable and see which version is running.

What to do

1. Run (curl asks for the password of the nexus user):

```
curl -i -u nexus http://localhost:8006/service/rest/v1/status
```

```
mahnoor-cs6767@ubuntu:~/DevOps$ curl -i -u nexus http://localhost:8006/service/rest/v1/status
Enter host password for user 'nexus':
HTTP/1.1 200 OK
Server: Nexus/3.98.0-07 (COMMUNITY)
Vary: Accept-Encoding
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src http: data: blob: 'unsafe-inline'; script-src http: 'unsafe-inline' 'unsafe-eval'
Content-Length: 0

mahnoor-cs6767@ubuntu:~/DevOps$
```

Figure 2: Nexus status endpoint returning 200 OK

What to look for

- "HTTP/1.1 200 OK" means the server is up and ready.
- The Server header shows Nexus/3.98.0-07 (COMMUNITY), the Community Edition.
- The password is typed at the prompt, so it is not stored in the command.

Step 3: List all repositories as an administrator

Nexus organizes artifacts into repositories. There are three kinds: hosted (stores your own files), proxy (caches files from another server) and group (combines several repositories behind one address).

What to do

1. Ask the REST API for the list of repositories with an account that has full access:

```
curl -u admin -X GET
"http://localhost:8006/service/rest/v1/repositories"
```

The start of this output has scrolled off the screenshot, so the exact command and account are not visible. This is the same request used in Step 4 with an administrator account.

What to look for

- The visible part lists maven-public (type group), nuget-hosted (hosted), nuget.org-proxy (proxy) and nuget-group (group).
- A proxy repository has a remote URL, for example https://api.nuget.org/v3/index.json, from which it fetches and caches files.
- Each repository shows its format, URL, whether it is online and its size.



Figure 3: Repositories visible to an administrator (end of the list)

Step 4: List the repositories as the nexus user

The nexus user is a restricted account for publishing. Run the same request as this user to see which repositories it is allowed to see.

What to do

1. Run (the password is entered at the prompt):

```
curl -u nexus -X GET "http://localhost:8006/service/rest/v1/repositories"
```

```
mahnoor-cs6767@ubuntu:~/IdeaProjects/java-maven-app$ curl -u nexus -X GET "http://localhost:8006/service/rest/v1/repositories"
Enter host password for user 'nexus':
[ {
  "name" : "maven-snapshots",
  "format" : "maven2",
  "type" : "hosted",
  "url" : "http://localhost:8006/repository/maven-snapshots",
  "online" : true,
  "size" : 0,
  "attributes" : { }
} ]mahnoor-cs6767@ubuntu:~/IdeaProjects/java-maven-app$
```

Figure 4: Repositories visible to the nexus user

What to look for

- Only one repository is returned: maven-snapshots (format maven2, type hosted, online).
- Compared with Step 3, this shows that the user's permissions limit what it can see, which is the intended setup for a publishing account.
- The repository's size is 0, because nothing has been uploaded yet.

Step 5: Confirm that the maven-snapshots repository is empty

Before publishing anything, check the repository in the Nexus web interface.

What to do

1. Open http://localhost:8006 in a browser, log in, and choose Browse, then maven-snapshots.

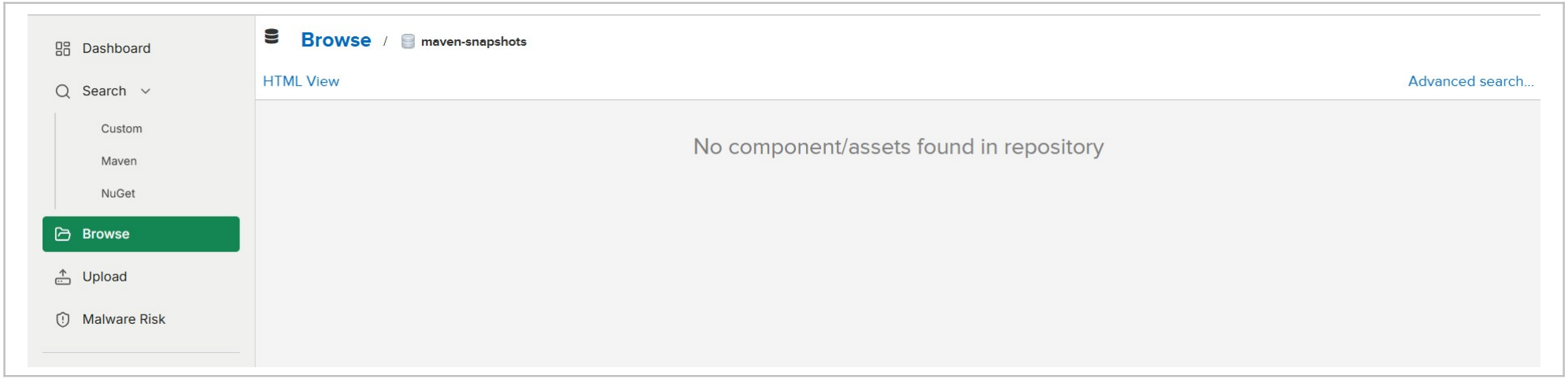


Figure 5: Empty maven-snapshots repository

What to look for

- The page says “No component/assets found in repository”.
- The left menu includes Search, Browse and Upload.

Part 2: Publish a Gradle Artifact to Nexus (java-app)

Step 6: Review the publishing settings in build.gradle

For Gradle to upload a file to Nexus, the project needs the maven-publish plugin and a repository definition that points to Nexus.

What to do

1. In the java-app project, print the build file:

```
cd ~/IdeaProjects/java-app
cat build.gradle
```

What to look for

- The plugins block now includes 'maven-publish'.
- The publications block defines a Maven publication that uploads build/libs/my-app-$version.jar as a jar.
- The repositories block points to the Nexus repository at http://localhost:8006/repository/maven-snapshots/. allowInsecureProtocol = true is needed because the address uses http instead of https.
- The credentials use project.repoUser and project.repoPassword, so the user name and password come from Gradle properties and are not written in the file. The screenshot ends inside the credentials block.

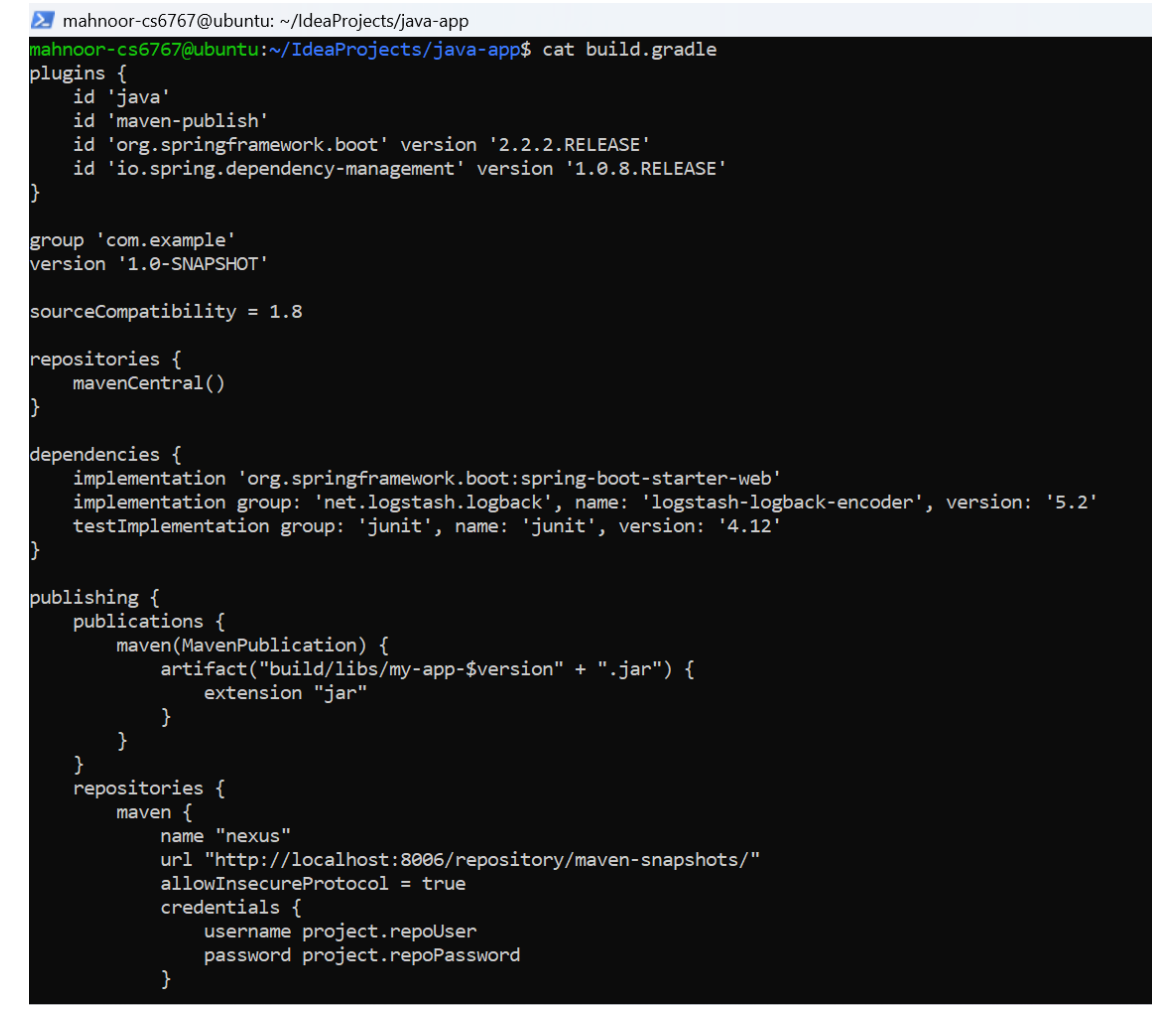


Figure 6: build.gradle with the Nexus publishing configuration

Step 7: Build the Gradle project

Before publishing, check that the project builds correctly from a clean state.

What to do

1. Run:

```
./gradlew clean build
```



Figure 7: Clean Gradle build

What to look for

- BUILD SUCCESSFUL in 13s with 4 actionable tasks executed.
- The message about incompatible or stopped daemons is informational: Gradle simply starts a new daemon.

Step 8: Publish the artifact to Nexus

Adding the publish task uploads the built JAR and its metadata to the Nexus repository.

What to do

1. Run:

```
./gradlew clean build publish
```

How the repoUser and repoPassword values were supplied (for example in gradle.properties) is not visible in the screenshots.

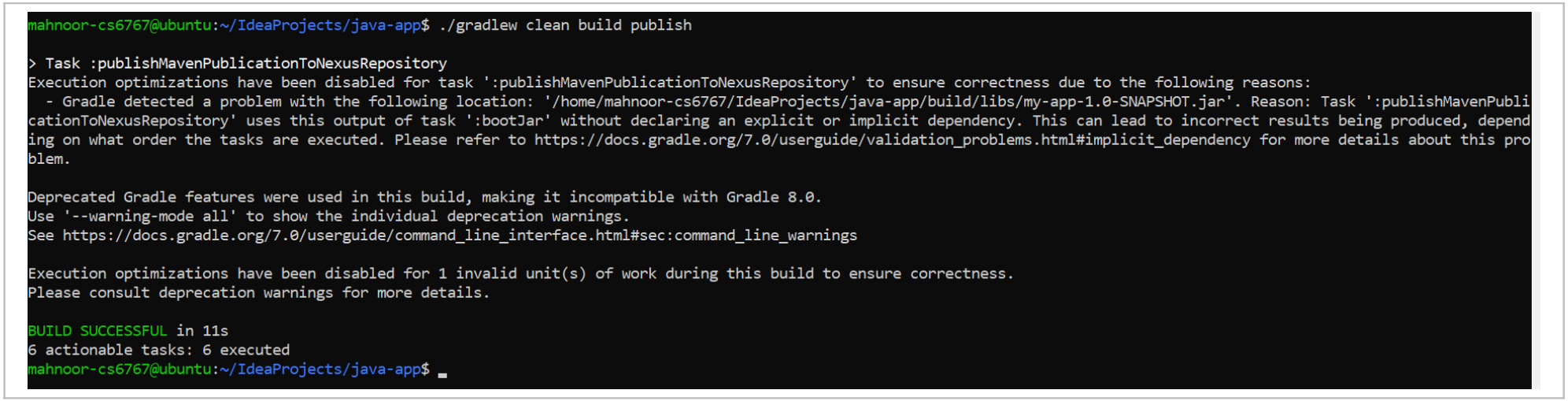


Figure 8: Gradle publishing to the Nexus repository

What to look for

- The task :publishMavenPublicationToNexusRepository runs, which is the upload to Nexus.
- Gradle prints a warning that the publish task uses the output of :bootJar without a declared dependency, so some optimizations were turned off. This is a warning about the build script and did not stop the upload.
- BUILD SUCCESSFUL in 11s with 6 actionable tasks executed.

Step 9: Check the Gradle artifact in Nexus

Open the repository in the web interface to see the uploaded files.

What to do

1. In Nexus, choose Browse, then maven-snapshots, and expand com > example > my-app > 1.0-SNAPSHOT.

What to look for

- The artifact is stored under com/example/my-app/1.0-SNAPSHOT in a folder named 1.0-20261108.115411-1.
- That folder name is a timestamp, and the time in it (11:54:11) is in UTC, which is five hours behind Pakistan time.
- The uploaded files are the JAR, the POM, and checksum files (md5, sha1, sha256 and sha512) that let tools verify the files.

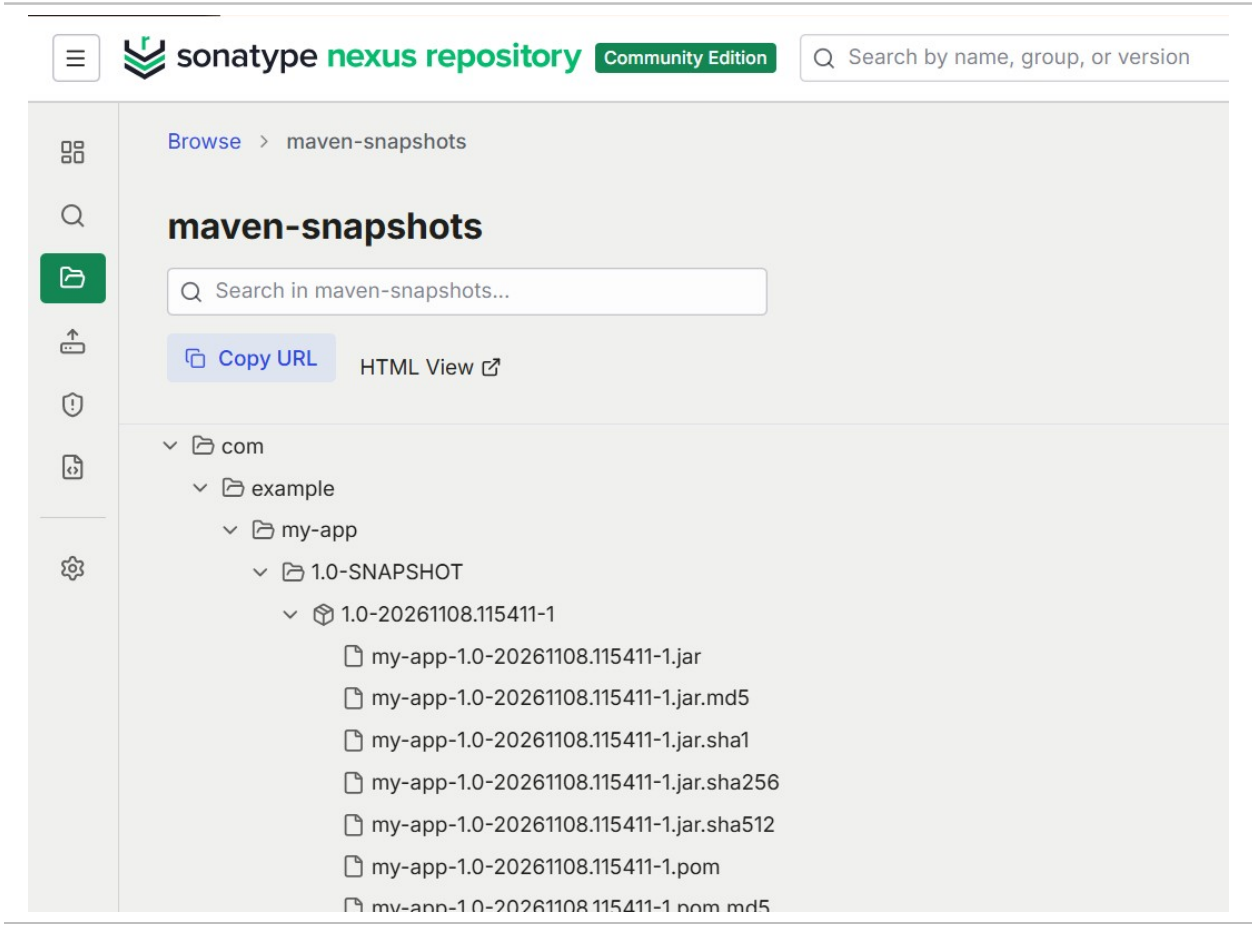


Figure 9: my-app snapshot uploaded to maven-snapshots

Step 10: List the repository's components with the REST API

Everything shown in the web interface is also available through the REST API. This request lists the components in maven-snapshots.

What to do

1. Run (the password is entered at the prompt):

```
curl -u nexus -X GET "http://localhost:8006/service/rest/v1/components?repository=maven-snapshots"
```

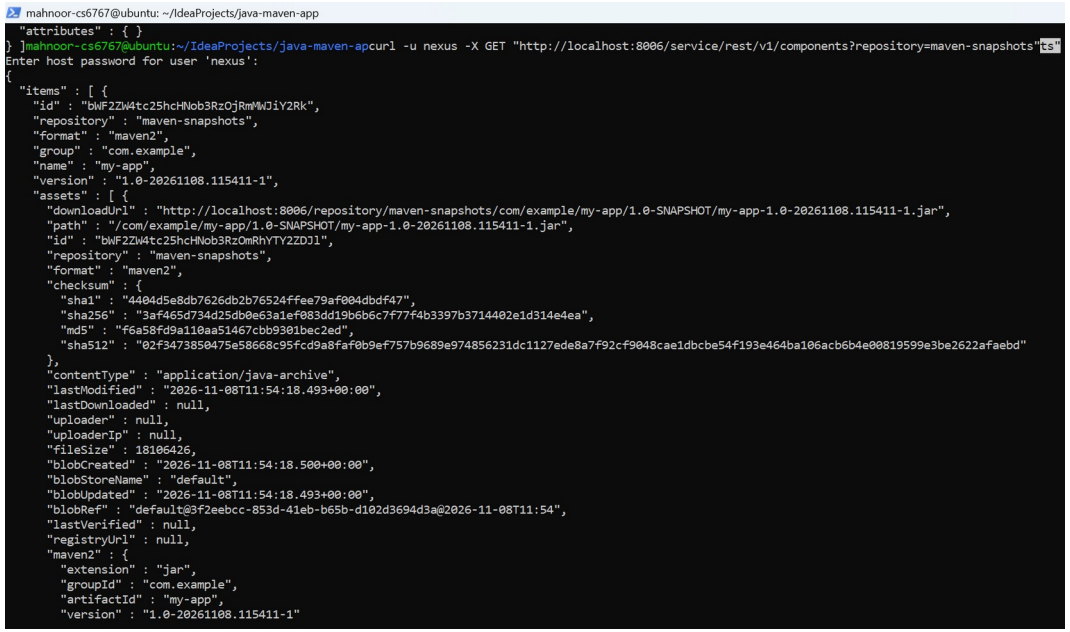


Figure 10: Components in maven-snapshots, returned as JSON

What to look for

- Exactly one component is listed: group com.example, name my-app, version 1.0-20261108.115411-1. The Maven artifact is not published yet, which matches the order of the lab.
- The JAR asset has a download URL, a content type of application/java-archive and a file size of 18,106,426 bytes (about 18.1 MB).
- It is stored in the blob store named default, and its checksums (sha1, sha256, md5, sha512) are listed.

Step 11: Get the details of one component

Every component has an ID. This command first finds the ID of the first component in the repository and then asks Nexus for its details.

What to do

1. Run (replace <password> with the user's password):

```
COMP_ID=$(curl -s -u nexus:<password> "http://localhost:8006/service/rest/v1/components?repository=maven-snapshots" |
grep -m 1 '"id"' | cut -d '"' -f 4) && curl -u nexus -X GET "http://localhost:8006/service/rest/v1/components/$COMP_ID"
```

In the screenshot the password is written in plain text inside the command. That is convenient in a lab, but in real use you should avoid it and use a prompt or an access token instead.

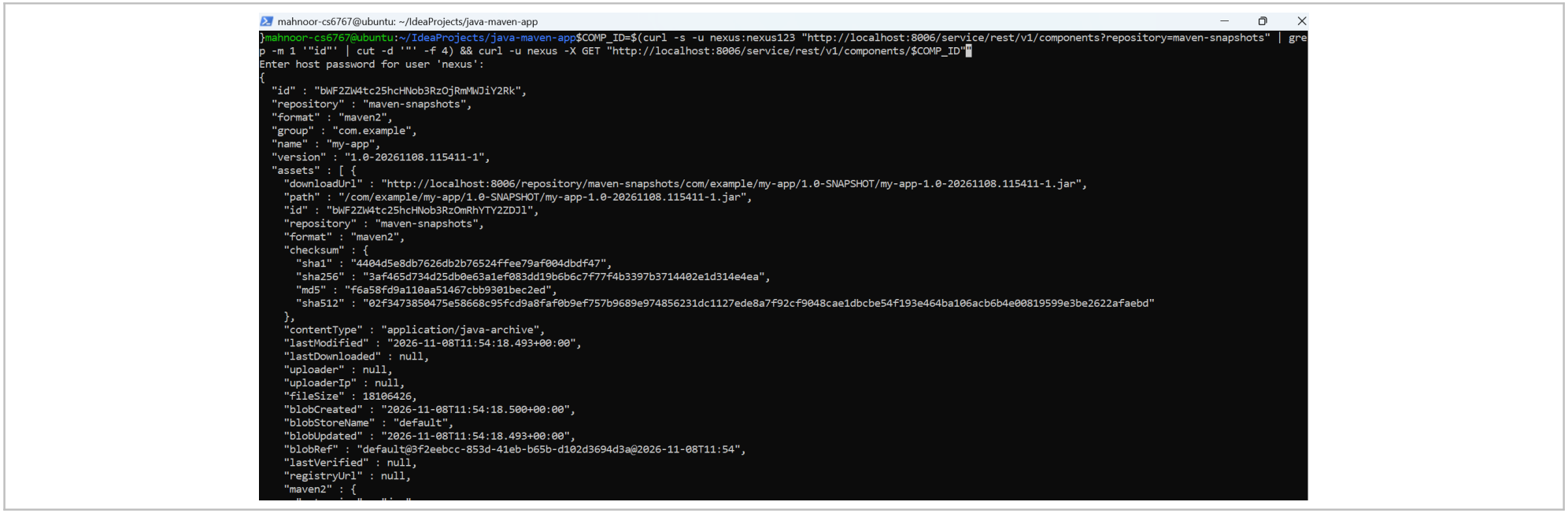


Figure 11: Details of a single component, found by its ID

What to look for

- The first command extracts the component ID with grep and cut and stores it in COMP_ID. The second command uses that ID in the URL.
- The returned ID matches the my-app component from the previous step.
- The output lists every asset of the component, including the POM checksum files (the last one visible is pom.sha512, 128 bytes).

Part 3: Publish a Maven Artifact to Nexus (java-maven-app)

Step 12: Open the java-maven-app project

Now publish the Maven project from Lab 05 to the same repository.

What to do

1. Go to the project folder and list its files:

```
cd ~/IdeaProjects/java-maven-app
pwd && ls -la
```

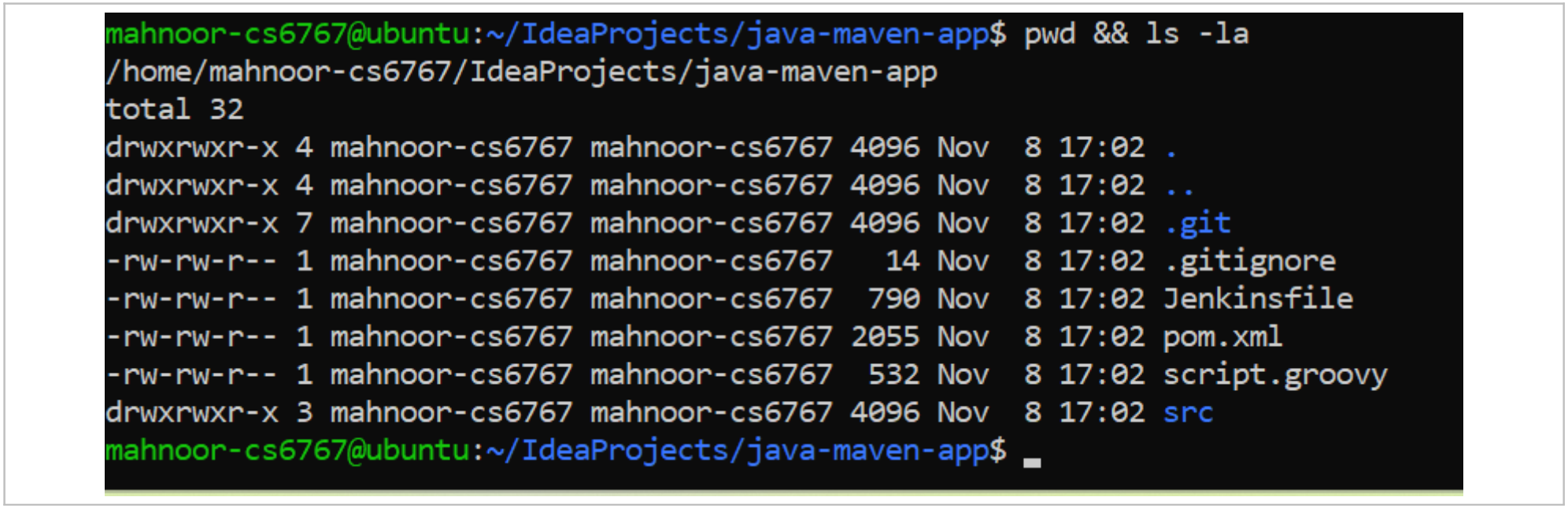


Figure 12: Files in the java-maven-app project

What to look for

- The project contains .git, .gitignore, Jenkinsfile, pom.xml, script.groovy and the src folder.
- All the entries are dated Nov 8 at 17:02, which shows that the project was freshly cloned.

Step 13: Add the distributionManagement section to pom.xml

Maven uses the distributionManagement section of pom.xml to know where to upload the artifact when you run mvn deploy.

What to do

1. Add the following block at the end of pom.xml, just before </project>:

```
<distributionManagement>
    <snapshotRepository>
        <id>nexus-snapshots</id>
        <url>http://localhost:8006/repository/maven-
snapshots/</url>
    </snapshotRepository>
</distributionManagement>
```

2. Print the end of the file to check it:

```
cat pom.xml
```

The command is not visible in the screenshot; it shows the last part of pom.xml.

What to look for

- The snapshotRepository has the id nexus-snapshots and the URL of the maven-snapshots repository in Nexus.
- Maven looks up the login details for this id in its settings file (settings.xml), which is not shown in the screenshots.
- The dependencies above it are spring-boot-starter-web 2.3.4.RELEASE, junit 4.13.1 (test scope) and logstash-logback-encoder 6.4.



Figure 13: pom.xml with the Nexus distributionManagement section

Step 14: Build the JAR with Maven

Package the application so that there is a JAR file to upload.

What to do

1. Run the package goal, then list the target folder:

```
mvn package
ls -la target/
```

The mvn package command is not visible in the screenshot; ls -la target/ is.

What to look for

- “Changes detected - recompiling the module!” appears because the source files are newer than the last build.
- The jar plugin builds java-maven-app-1.1.0-SNAPSHOT.jar and Spring Boot then repackages it.
- BUILD SUCCESS in 3.932 seconds, finished at 17:07:58.
- In the target folder, the JAR is 17,200,366 bytes (about 17.2 MB). The .jar.original file is the small JAR from before repackaging.

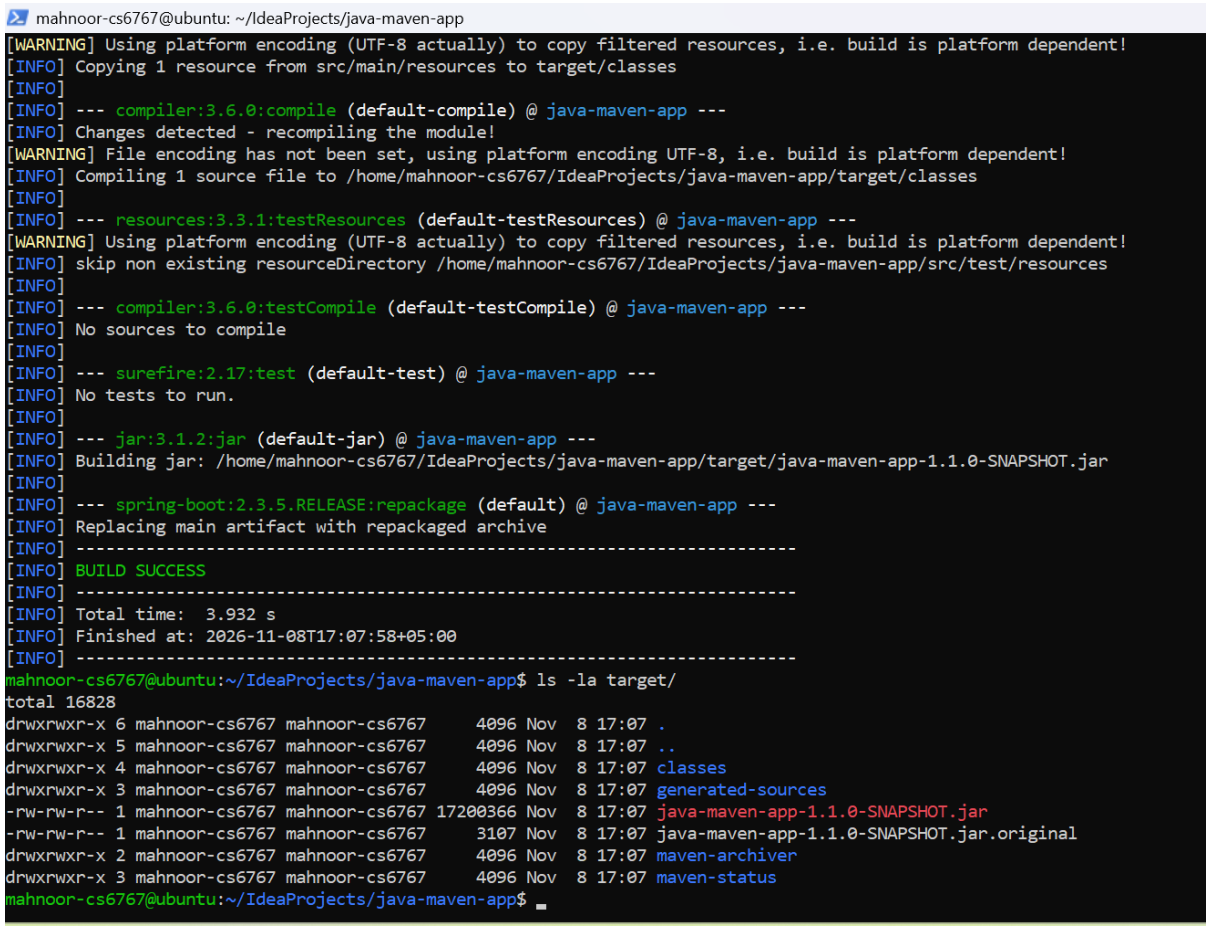


Figure 14: Maven package build and the generated JAR

Step 15: Deploy the artifact to Nexus

The deploy goal installs the artifact in your local Maven repository and then uploads it to the repository defined in distributionManagement.

What to do

1. Run:

```
mvn deploy
```

The command is not visible in the screenshot; this is the standard command that produces this output.

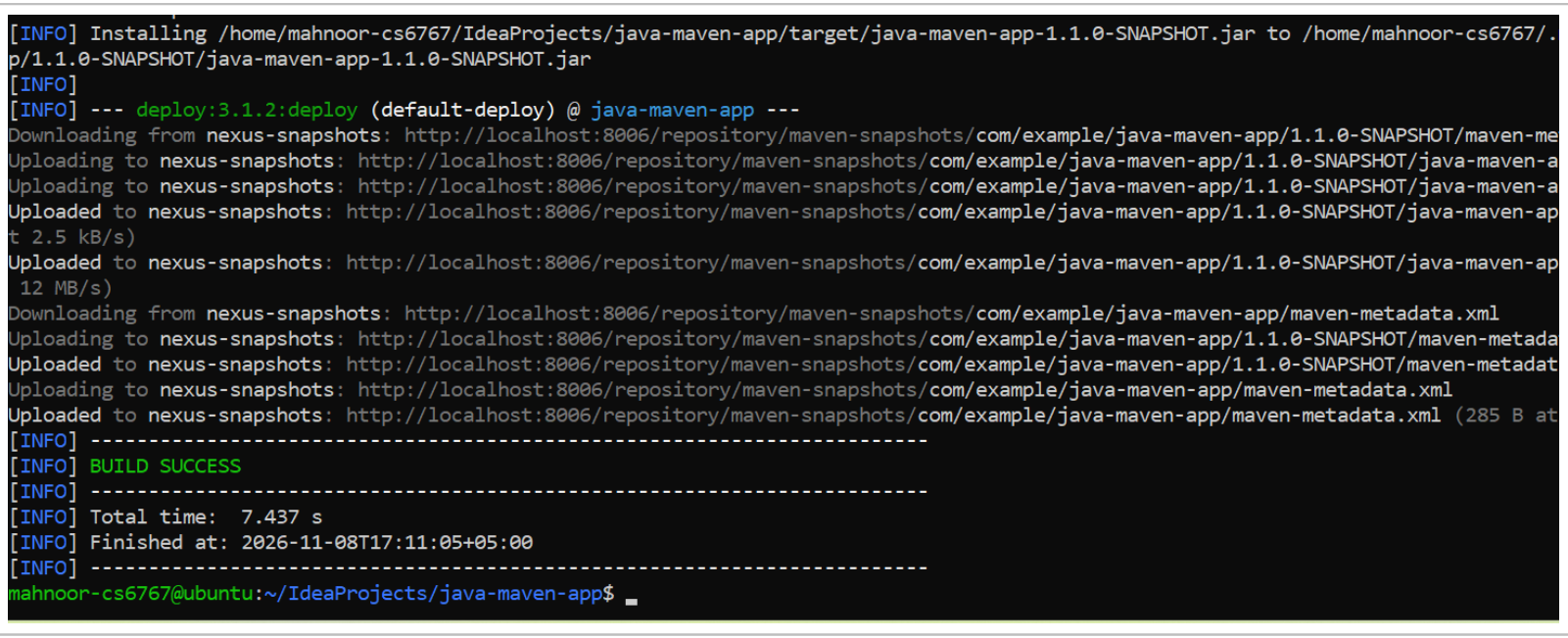


Figure 15: Maven deploying java-maven-app to Nexus

What to look for

- "Installing … to …" is the copy into the local repository.
- The deploy plugin then prints "Uploading to nexus-snapshots" and "Uploaded to nexus-snapshots" for the JAR, the POM and the maven-metadata.xml files.
- BUILD SUCCESS in 7.437 seconds, finished at 17:11:05.

Step 16: Check the Maven artifact in Nexus

Refresh the repository in the web interface to see the new upload.

What to do

1. In Nexus, choose Browse, then maven-snapshots, and expand com > example > java-maven-app > 1.1.0-SNAPSHOT.

What to look for

- The artifact is under com/example/java-maven-app/1.1.0-SNAPSHOT in the folder 1.1.0-20261108.121057-1 (the time is UTC).
- The folder contains the JAR and the POM with their md5 and sha1 checksum files.
- maven-metadata.xml, which lists the available versions, is stored one level above.

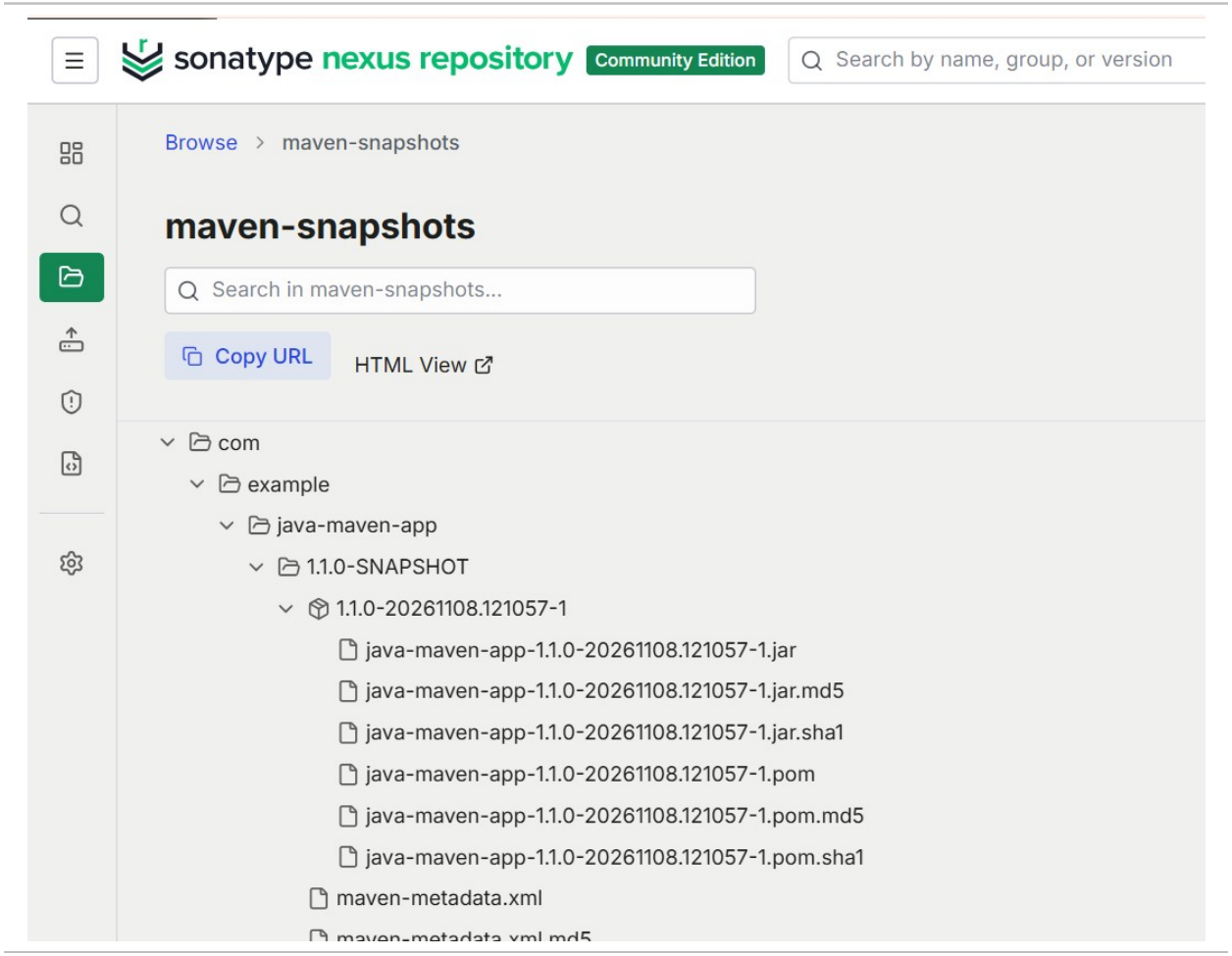


Figure 16: java-maven-app snapshot uploaded to maven-snapshots

Part 4: Blob Stores and a Docker Repository

Step 17: Look inside the default blob store

A blob store is the place on disk where Nexus keeps the actual files of all repositories. Open a shell inside the container to see how it is laid out.

What to do

1. Open a shell in the Nexus container and look at the blobs folder:

```
docker exec -it nexus /bin/sh
cd sonatype-work/nexus3/blobs
pwd
ls
ls default/content
```

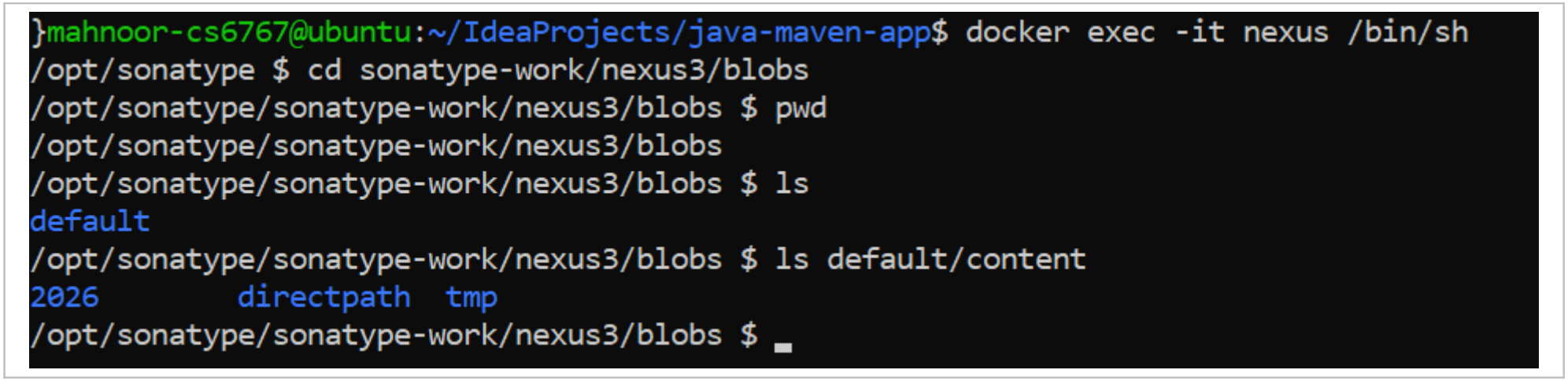


Figure 17: Default blob store inside the Nexus container

What to look for

- The blobs folder is /opt/sonatype/sonatype-work/nexus3/blobs and currently holds only one blob store, default.
- The content folder of the default store contains the folders 2026, directpath and tmp.

Step 18: Create a new blob store named mystore

Separate blob stores let you keep different repositories on different storage locations or disks.

What to do

1. In Nexus, go to Settings > Repository > Blob Stores and click Create Blob Store. Create a File blob store named mystore.

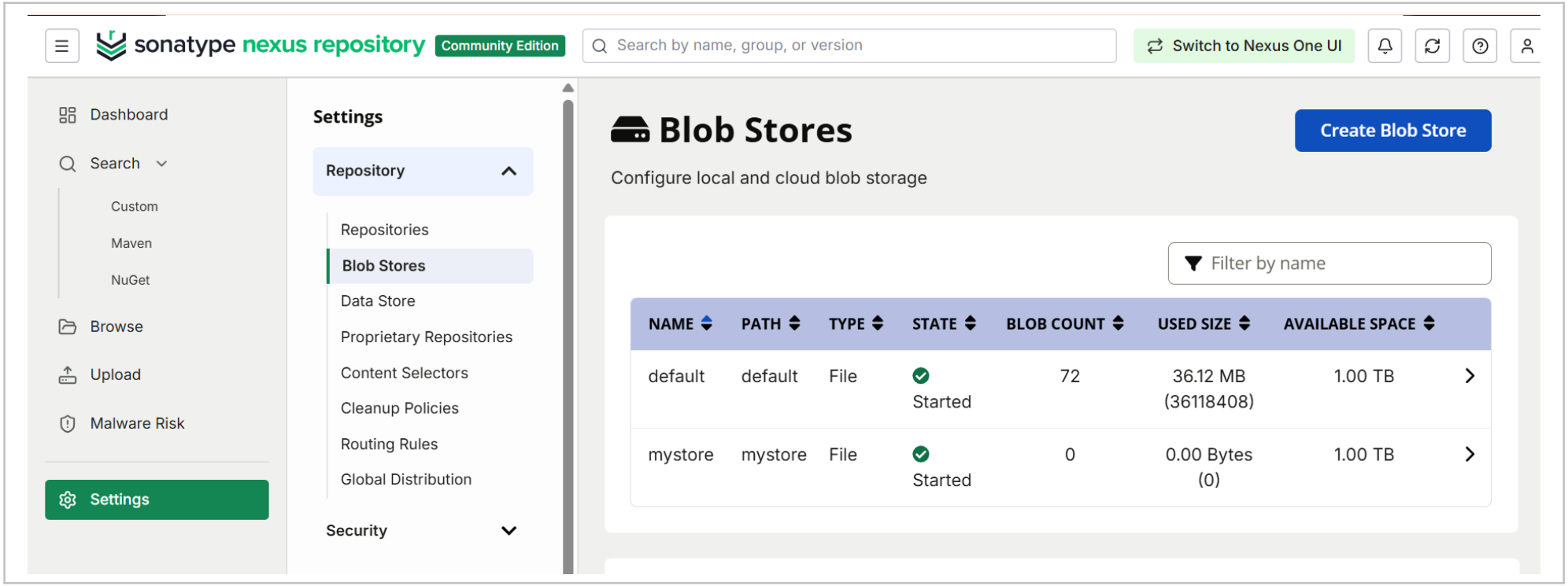


Figure 18: Blob Stores page showing default and mystore

What to look for

- Two blob stores are listed, both with state “Started” and 1.00 TB of available space.
- default holds 72 blobs (36.12 MB), because it already stores the uploaded artifacts and Nexus’s own files.
- mystore is new and empty (0 blobs, 0 bytes). The creation dialog itself is not part of the screenshots.

Step 19: Verify the new blob store on disk

Go back to the shell in the container to confirm that Nexus created a folder for the new blob store.

What to do

1. In the same shell, run:

```
pwd
ls
ls mystore
exit
```

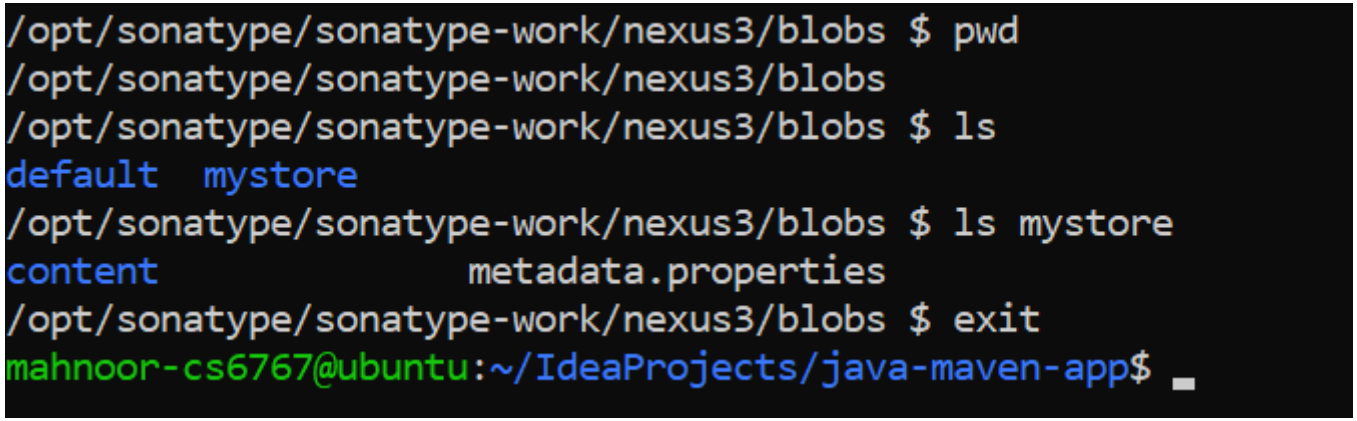


Figure 19: The mystore folder created inside the blobs folder

What to look for

- ls now shows two folders, default and mystore.
- mystore contains a content folder and a metadata.properties file.
- exit closes the shell and returns you to the normal prompt.

Step 20: Create a Docker repository that uses mystore

A hosted Docker repository stores Docker images pushed by your own projects. This one is set up to keep its data in the new blob store.

What to do

1. In Nexus, go to Settings > Repository > Repositories, click Create repository and choose docker (hosted). Use these settings:

```
Name: docker-hosted
HTTP connector port: 8007
Blob store: mystore
```

2. Open the repository to check its settings (shown here: the settings page of the created repository).

What to look for

- The repository is called docker-hosted, with format docker and type hosted, and it is online.
- An HTTP connector on port 8007 lets Docker clients connect directly. This is the port that was published when the container was started (Step 1).
- Under Storage, the blob store is mystore. Other settings stay at their defaults, including “Allow redeploy”.

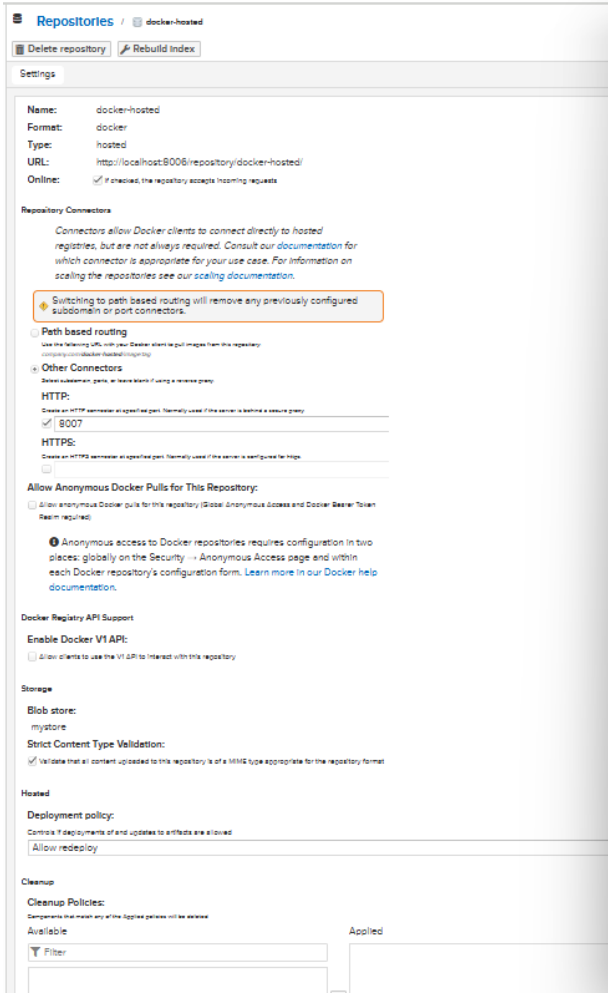


Figure 20: Settings of the docker-hosted repository

Part 5: Cleanup Policies and Blob Store Compaction

Step 21: Create a cleanup policy

Over time, repositories collect old snapshots that use up disk space. A cleanup policy describes which components should be removed automatically.

What to do

1. In Nexus, go to Settings > Repository > Cleanup Policies and click Create Cleanup Policy. Fill in:

```
Name: maven
Format: maven2
Release Type: Pre-Releases/Snapshots
Asset Name Matcher: .*
```

2. Choose maven-snapshots in the Preview Repository list and click Preview, then click Save.

What to look for

- The policy applies to the maven2 format and to Pre-Releases/Snapshots.
- The criterion is an Asset Name Matcher with the pattern .*, which matches every file name. Component Age and Component Usage are not used.
- The preview lists the two components that would be removed: java-maven-app 1.1.0-20261108.121057-1 and my-app 1.0-20261108.115411-1.
- Because .* matches everything, this policy removes all snapshots. That is fine for a lab but too aggressive for real use, where you would normally limit by age or usage.

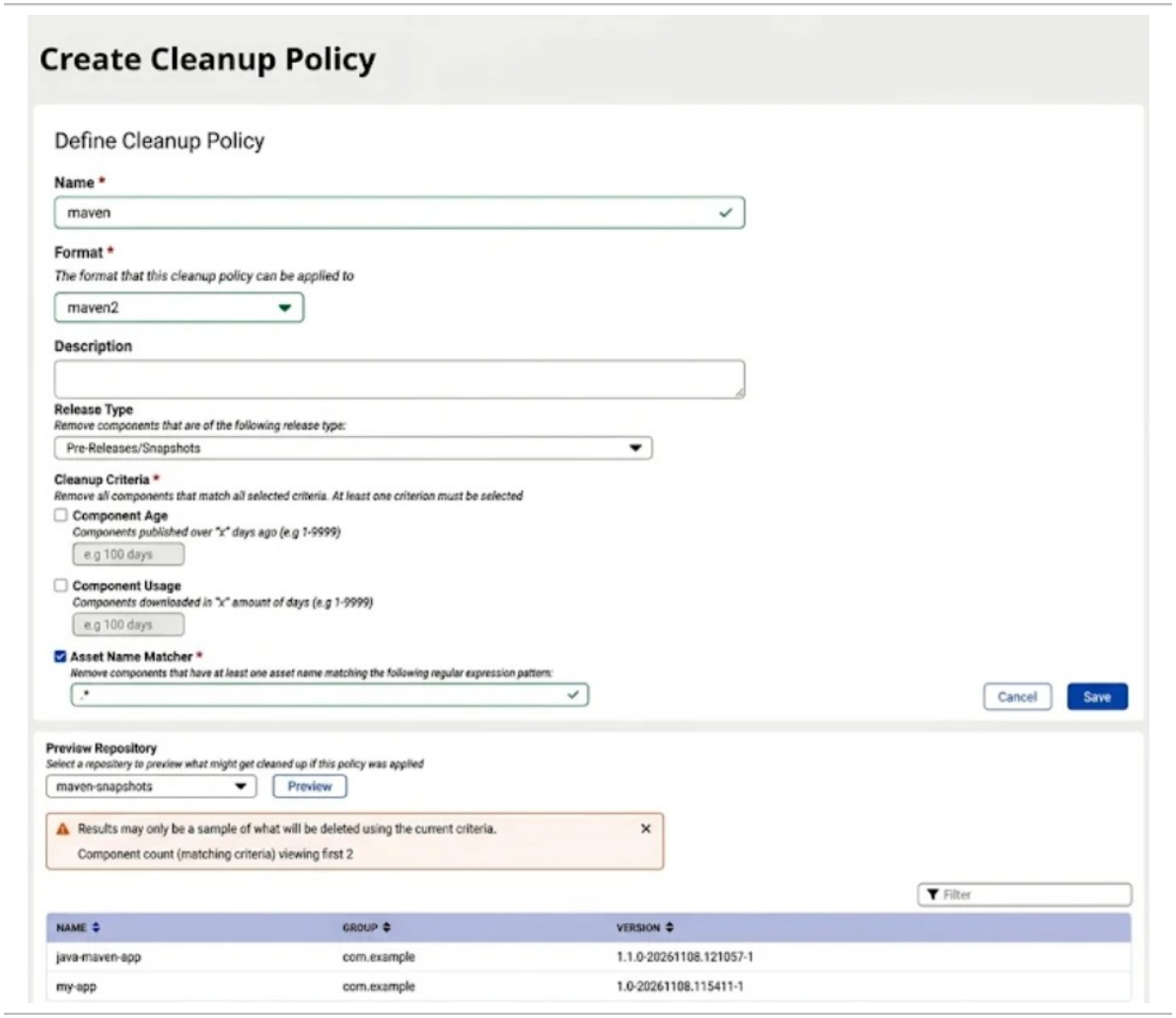


Figure 21: Create Cleanup Policy form with a preview of matching components

Step 22: Apply the cleanup policy to maven-snapshots

A policy only takes effect when it is attached to a repository.

What to do

1. Go to Settings > Repository > Repositories, open maven-snapshots and scroll to the Cleanup section. Move maven from Available to Applied, then save.

What to look for

- maven now appears in the Applied list.
- The note under “Cleanup Policies” says that components matching any applied policy will be deleted.

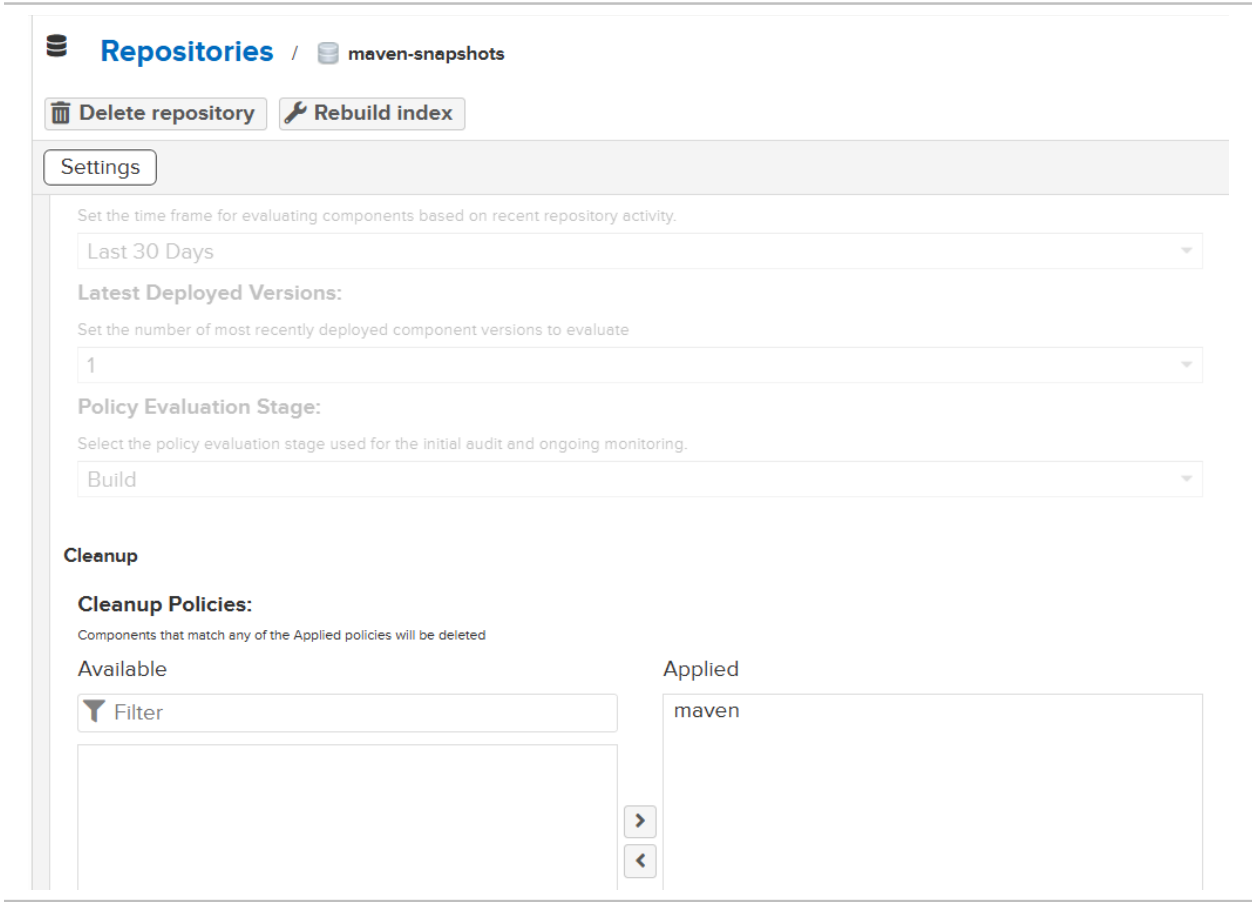


Figure 22: The maven cleanup policy applied to maven-snapshots

Step 23: Run the cleanup task

Cleanup policies are carried out by a scheduled task called Cleanup service. You can also start it by hand.

What to do

1. Go to Settings > System > Tasks and open the Cleanup service task. Its summary shows the schedule and the last run:

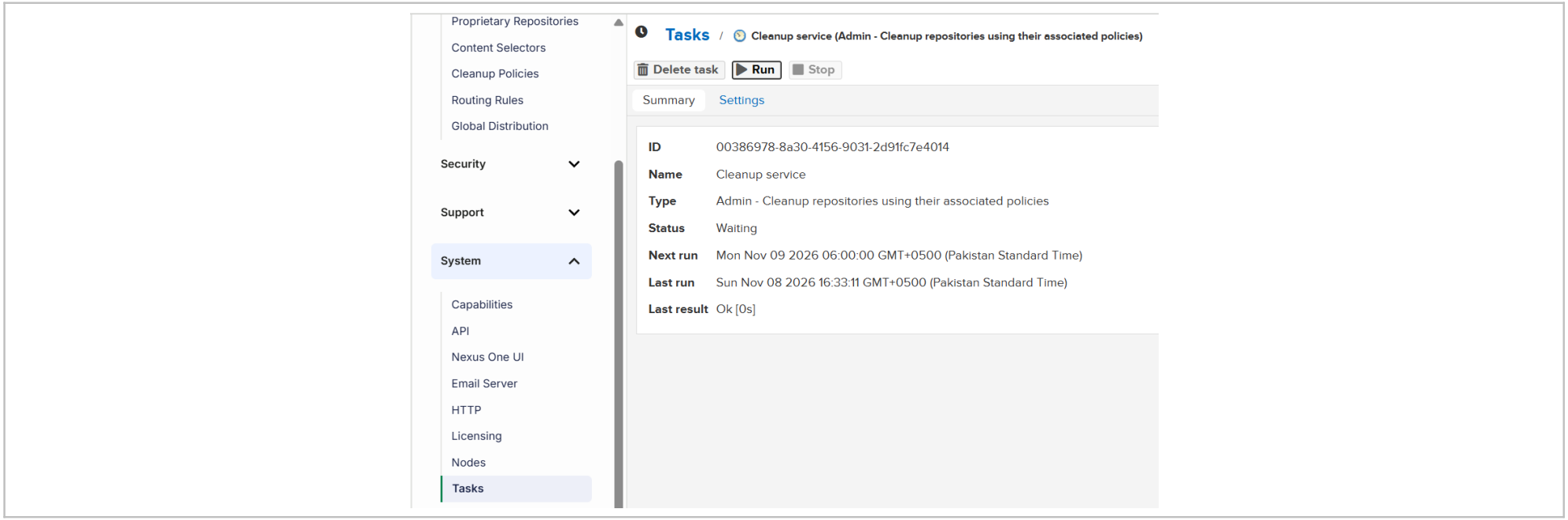


Figure 23: Cleanup service task before the manual run

2. Click Run, then refresh the summary:

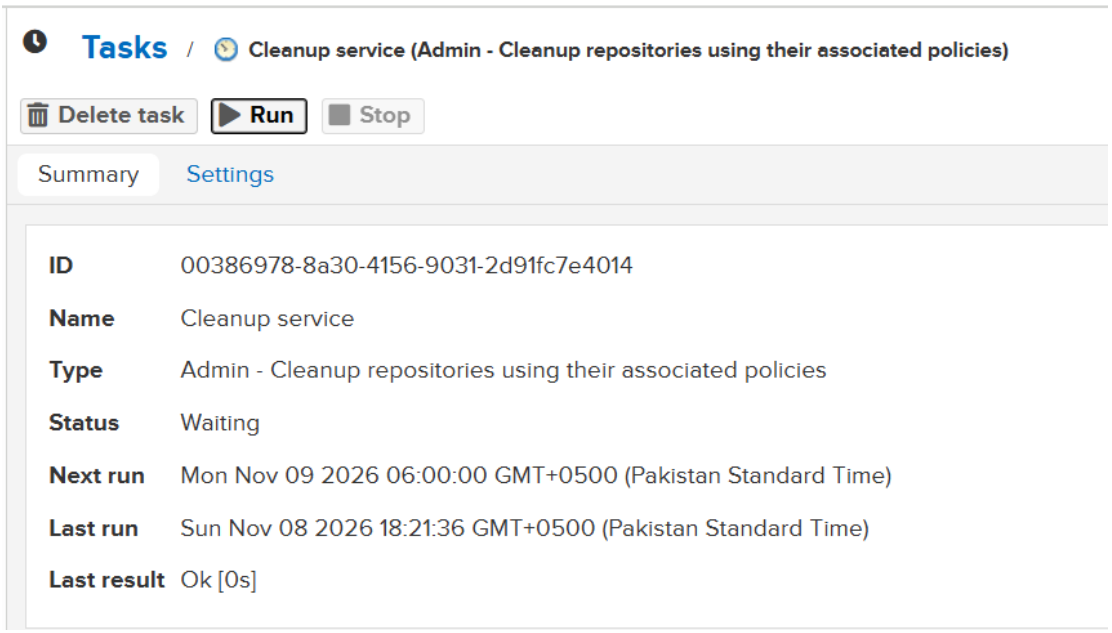


Figure 24: Cleanup service task after the manual run

What to look for

- The task type is “Admin - Cleanup repositories using their associated policies”, with the status Waiting and the next scheduled run on Mon Nov 09 at 06:00.
- Before the manual run, the last run was Sun Nov 08 at 16:33:11 with the result Ok.
- After clicking Run, the last run changes to Sun Nov 08 at 18:21:36, again with the result Ok [0s].

Step 24: Check the blob store before compaction

Deleting components in Nexus does not free disk space straight away. The files are only marked as deleted and are removed later when the blob store is compacted. First note the current numbers.

What to do

1. Go to Settings > Repository > Blob Stores.

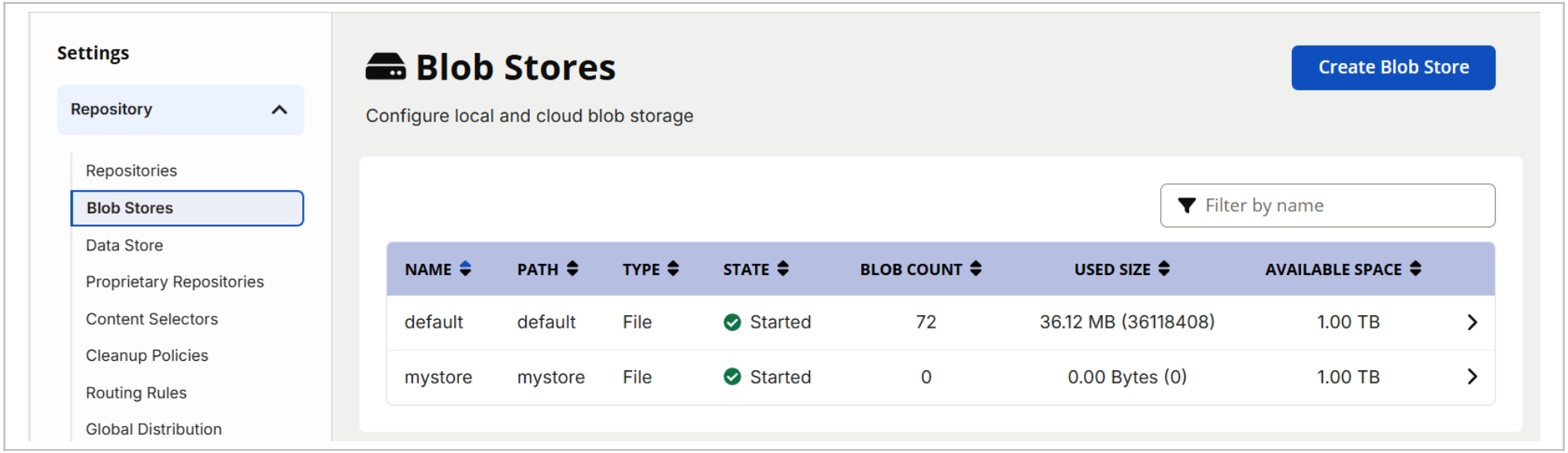


Figure 25: Blob stores before compaction

What to look for

- The default blob store still holds 72 blobs and 36.12 MB, even though the snapshots were cleaned up.
- mystore is still empty.

Step 25: Create the compaction task

The Compact blob store task permanently removes the deleted files and gives the disk space back.

What to do

1. Go to Settings > System > Tasks, click Create task and choose “Admin - Compact blob store”. Fill in the settings:

```
Task name: compact-store
Send notification on: Failure
Blob store: default
Blobs older than (days): 0
Task frequency: Daily
Start date: 11/08/2026
Time to run: 06:30
```

What to look for

- The task works on the default blob store.
- “Blobs older than: 0” means deleted blobs are removed immediately instead of being kept for some days.
- It is scheduled daily at 06:30 Pakistan time, and an email notification is only sent if the task fails (no email address is entered).

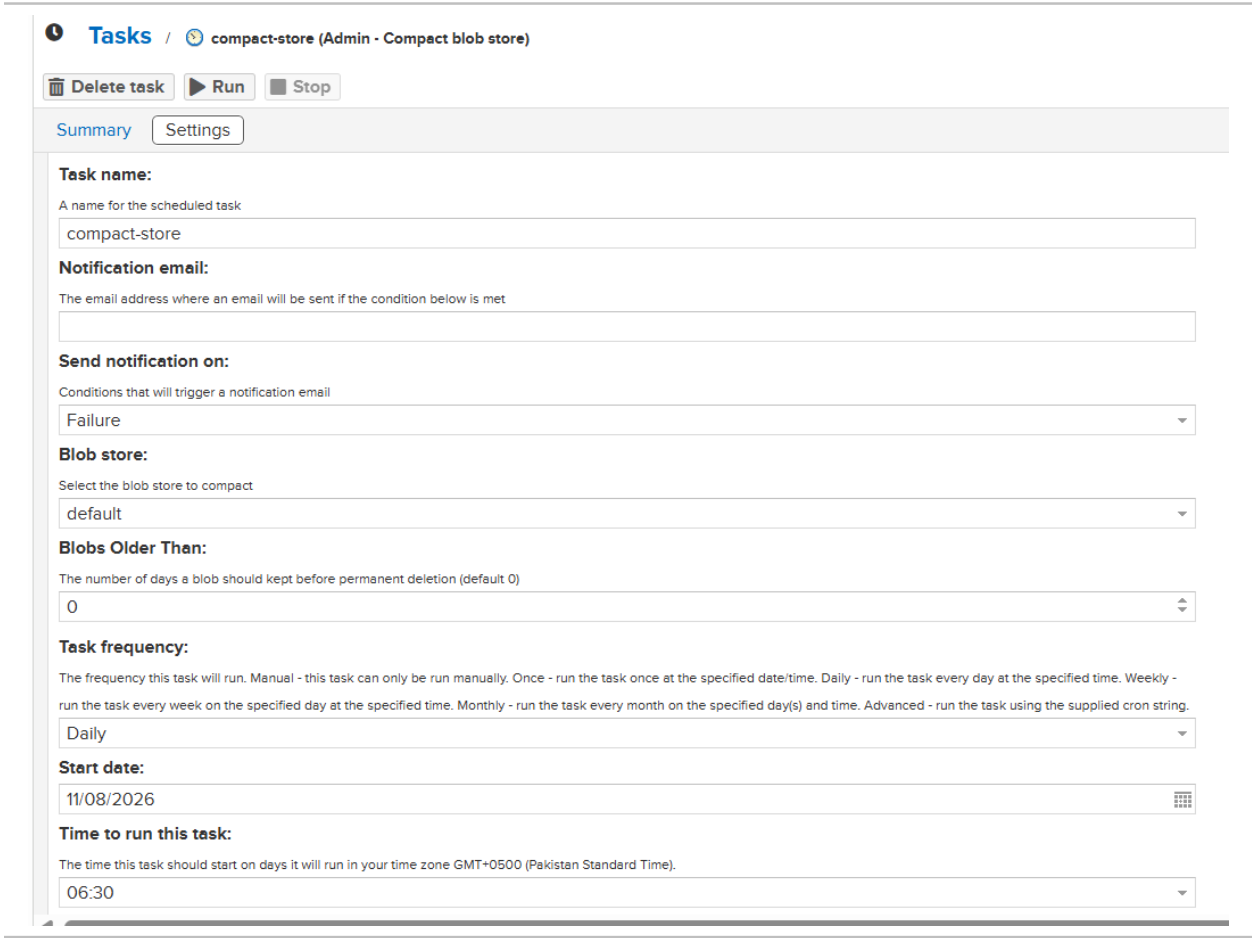


Figure 26: Settings of the compact-store task

Step 26: Run the compaction task

Start the task by hand instead of waiting for tomorrow's schedule.

What to do

1. Open the compact-store task and click Run.

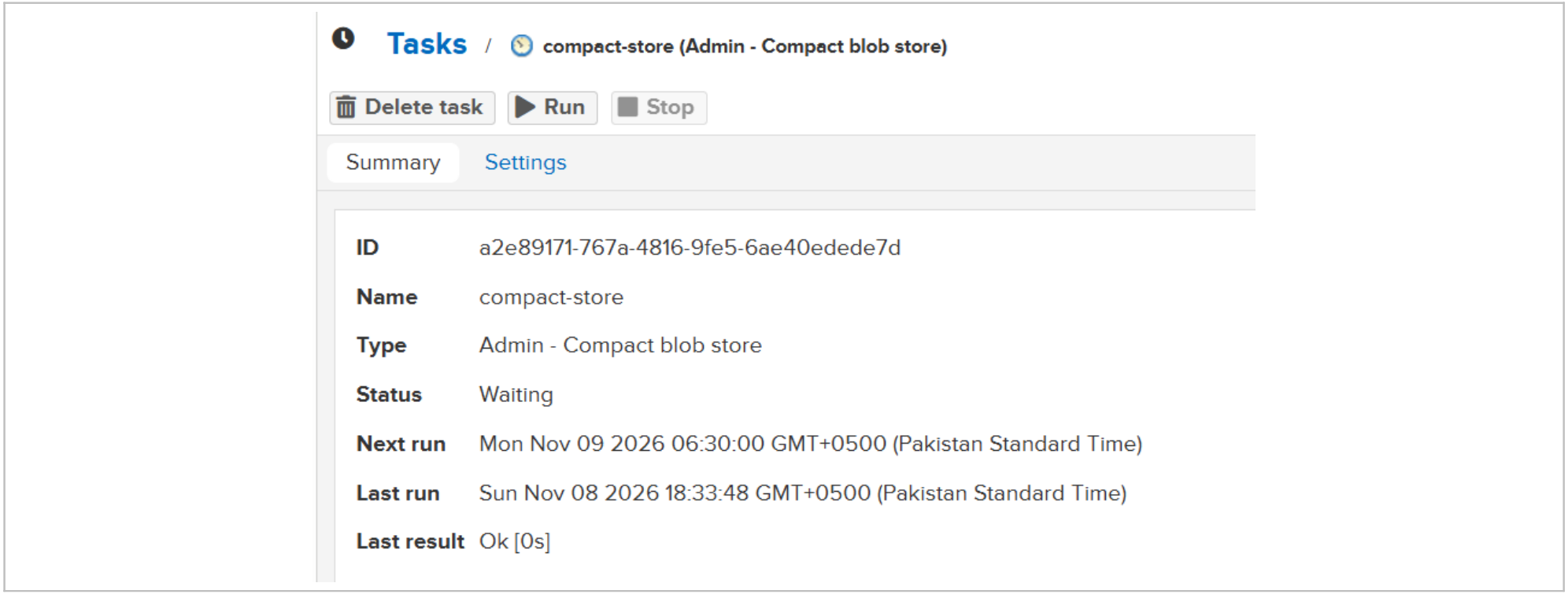


Figure 27: compact-store task after running

What to look for

- The task type is "Admin - Compact blob store".
- The last run is Sun Nov 08 at 18:33:48 with the result Ok [0s].
- The next run is scheduled for Mon Nov 09 at 06:30.

Step 27: Check the blob store after compaction

Go back to the blob store list to see the effect of the cleanup and compaction.

What to do

1. Go to Settings > Repository > Blob Stores.

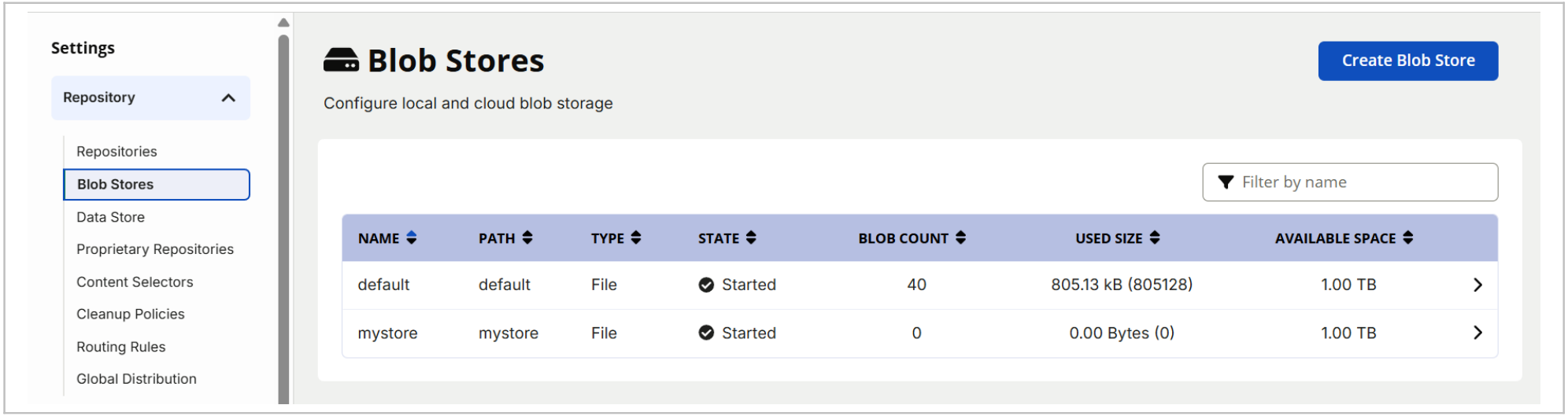


Figure 28: Blob stores after compaction

What to look for

- The default blob store dropped from 72 blobs and 36.12 MB to 40 blobs and 805.13 kB.
- The disk space used by the removed snapshots has now been given back.
- mystore is unchanged, because no repository has stored anything in it yet.

Summary of Results

Item	Details
Nexus web interface and API	http://localhost:8006 (Docker connector port 8007)
Gradle artifact	com.example:my-app 1.0-SNAPSHOT, published at 16:54 (JAR about 18.1 MB)
Maven artifact	com.example:java-maven-app 1.1.0-SNAPSHOT, deployed at 17:11 (JAR about 17.2 MB)
New blob store	mystore, used by the docker-hosted repository
Cleanup policy	maven (maven2, Pre-Releases/Snapshots, asset name .*), applied to maven-snapshots
Blob store default	Before compaction: 72 blobs, 36.12 MB. After compaction: 40 blobs, 805.13 kB

Conclusion

In this lab, you used Nexus Repository Manager as a central store for build artifacts. After checking that the Nexus container was running and that the restricted nexus user could see only the maven-snapshots repository, you published a Gradle artifact and a Maven artifact to it and verified both in the web interface and through the REST API. You then created a second blob store and a Docker repository that uses it. Finally, you set up a cleanup policy and a compaction task and saw the default blob store shrink from 36.12 MB to 805.13 kB. Together, these steps show how a repository manager stores, organizes and cleans up the artifacts that a build pipeline produces.